Assignment 1: Conceptual Architecture

A closer look at the architecture of WezTerm

Group 1

Authors: Matthew Hewlitt - matthew.hewlitt@queensu.ca, Miguel Pereira - 23htl1@queensu.ca, Jason Chen - jason.chen@queensu.ca, Jose Kerketta - 23brf3@queensu.ca, Stefan Ciofu - 23by38@queensu.ca, Andrew Nguyen - 23bl79@queensu.ca

0 Abstract

This document outlines the conceptual architecture of WezTerm, an open-source, cross-platform terminal emulator and multiplexer written primarily in Rust. Using the official documentation, the repository structure, package dependencies and two representative use cases, we propose an architecture of 12 components organized around a central Multiplexer, which maintains the state of every window, tab, pane and workspace and notifies other components when that state changes. The GUI Front End and the Launcher & CLI provide two ways of interacting with this state, Domains connect panes to local or remote programs, the Terminal Emulation Core interprets escape sequences and keeps screen state, and the PTY & Process Layer runs local programs. Rendering, font handling, operating-system integration, remote multiplexing, SSH and Lua-based configuration provide the remaining functionality.

From an architectural perspective, the dominant style is a repository with change notification, complemented by a client-server model for remote sessions and elements of a relaxed layered design. We concluded that the design demonstrates modularity and evolvability, and that the shared session model lets a graphical terminal and persistent remote sessions live in a single application. The main performance risk is the output path, which terminal programs can flood with data; the design addresses it with background reader and parser threads, output coalescing and delta-only updates for remote panes. Because WezTerm publishes no definitive architecture document, our component boundaries and style classifications are interpretations of the available evidence.

1 Introduction and Overview

WezTerm is an open-source, cross-platform terminal emulator and multiplexer that lets users interact with command-line programs through a graphical interface. Beyond displaying text, it organizes terminal sessions into panes, tabs, windows and workspaces, lets users customize its behaviour through Lua configuration files, and connects to remote environments through SSH and remote multiplexing. These features require coordination between user interaction, terminal emulation, process management, rendering and session management. WezTerm is therefore a useful subject for architectural analysis, because it combines the responsibilities of a traditional terminal emulator with those of a session manager and a remote-access tool.

The purpose of this report is to develop a conceptual architecture that explains WezTerm's major components, their responsibilities and the relationships between them. Rather than documenting every implementation detail, we focus on the structural decisions that enable WezTerm to support its primary functionality. Our analysis draws on the official documentation, the organization of the source repository, package dependency information and the behaviour of representative use cases. Since WezTerm does not provide a definitive architecture document, the proposed model is our interpretation of the available evidence rather than an official description of the system's design.

Our analysis identifies a repository with change notification as WezTerm's dominant architectural style. The Multiplexer maintains a shared session state and notifies interested components when it changes, so the GUI can update the display without managing every terminal's internal state. A

client-server architecture complements this design for remote sessions, and the layering of the components is relaxed rather than strict. This separation matters because the Multiplexer owns the organization of sessions, the Terminal Emulation Core owns the contents of each terminal, and the Renderer only draws what it is given. Keeping these responsibilities apart explains how WezTerm manages terminal interaction without requiring the GUI, the Multiplexer and the terminal emulator to perform the same tasks.

This report is structured both structurally and behaviourally. The architecture section explains how we derived the 12 components, presents the box-and-arrow diagram, justifies the architectural styles, compares WezTerm with Ghostty, kitty, tmux and Alacritty, and describes each subsystem and the rationale for its interactions. It then covers control flow and concurrency, evolution and the division of developer responsibilities. Later sections present the external interfaces and two use cases, tracing a command in a local pane and an attachment to a remote session over SSH, followed by a data dictionary, naming conventions, conclusions, lessons learnt and the report on our collaboration with an AI teammate.

2 Architecture

2.1 Derivation Process

We derived WezTerm's conceptual architecture in seven steps, moving from user-facing documentation to package structure and then testing the result against use cases and competing designs. Every component and dependency in our diagrams can be traced to a documentation page or a named package in the repository.

1) **Read the user documentation.** We started with the feature list, “What is a Terminal?”, Multiplexing, SSH and the Lua configuration reference [1-5]. This gave us WezTerm's functions and its user-visible concepts: panes, tabs, windows, workspaces and domains.
2) **Map concepts to packages.** The repository is split into more than 60 Rust packages (crates), most with a one-line description [6]. These descriptions and the contributor guide, which states that the terminal model is “agnostic of any windowing system” [7], gave us candidate subsystems.
3) **Group packages into components.** We merged helper packages (escape-sequence parsers, cell and colour types) into the component that owns them, giving 12 top-level components, within the 5-15 range we judged readable on one diagram.
4) **Derive dependencies.** For each component we recorded which others it calls or relies on, using the packages' declared dependencies as evidence. We removed dependencies that only share data types and kept those that reflect a request for a service. Because WezTerm publishes no architecture document, this step draws on package metadata as well as documentation.
5) **Identify styles.** We compared the structure against Garlan and Shaw's catalogue of styles [18], looking for a shared store with notifications, a network protocol between processes, and lower layers that work without the ones above them.
6) **Validate with use cases.** We walked two scenarios through the diagram: typing a command in a local pane, and attaching to a remote session over SSH. Any step with no box or arrow to

carry it showed a gap, which we fixed before freezing the diagram.

7) **Compare with other terminals.** We read the architecture overviews of Ghostty, kitty, tmux and Alacritty [8-11] to check whether our component boundaries are specific to WezTerm or common to all terminal emulators.

2.2 Conceptual Architecture

WezTerm's conceptual architecture has 12 components arranged around one hub, the Multiplexer, which holds every window, tab and pane. Three front ends reach it (the command line, the GUI and the remote mux), and it creates panes through Domains, each of which pairs a terminal model with a source of bytes.

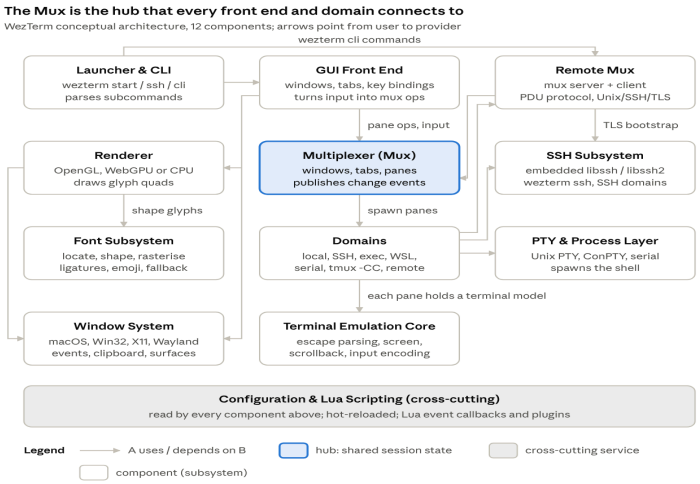


Figure 1. WezTerm conceptual architecture: 12 components, dependency arrows.

Component	Role
Launcher & CLI	Entry point; starts the GUI or runs WezTerm cli commands against a running instance [17]
GUI Front End	Windows, tabs, key bindings and overlays; turns user input into Multiplexer operations
Renderer	Draws pane contents with OpenGL, WebGPU or a CPU fallback, chosen by front_end [16]
Window System	One API over macOS, Windows, X11 and Wayland windows, input and clipboard
Font Subsystem	Locates, shapes and rasterises glyphs, with ligatures, emoji and fallback fonts [1]
Multiplexer	Shared store of windows, tabs, panes and workspaces; broadcasts every change [13]
Domains	Pluggable sources of panes: local, exec, WSL, serial, SSH, tmux and remote mux [3]
Terminal Emulation Core	Parses escape sequences and models the screen and scrollback; no GUI and no PTY [12]
PTY & Process Layer	Creates the pseudo-terminal (or Windows ConPTY) and spawns the shell [2]
Remote Mux	Mux server and client, speaking a versioned protocol over Unix sockets, SSH or TLS [3, 14]

SSH Subsystem	Embedded SSH client for WezTerm ssh, SSH domains and TLS bootstrap [4]
Configuration & Lua	Runs WezTerm.lua, reloads it on change, dispatches events and loads plugins [5, 15]

The same 12 components fall into five layers plus a cross-cutting configuration service. The Session layer is the boundary that every front end crosses to reach a running program.

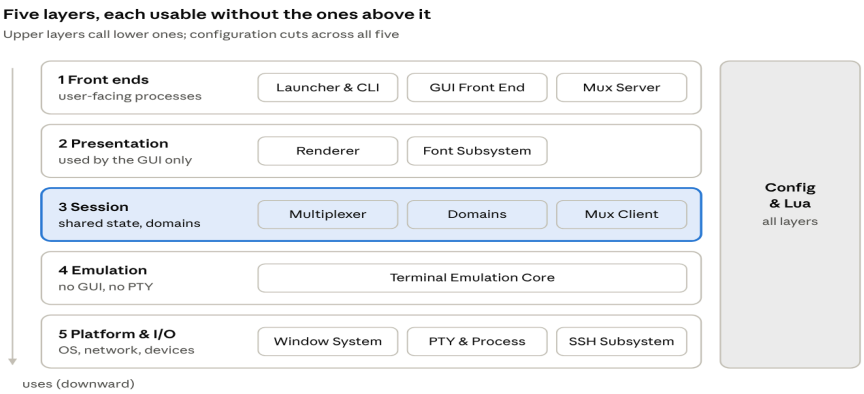


Figure 2. Layered view: 5 layers and a cross-cutting configuration service.

In Figure 2 the Remote Mux appears split across two layers: its server side is a front end, and its client side sits in the Session layer as a Domain. Presentation and Session are siblings under the GUI rather than stacked: the GUI uses both, but neither uses the other. The layering is therefore relaxed rather than strict, as discussed in the next section.

2.3 Architecture Style

WezTerm's overall style is a repository with change notification, centred on the Multiplexer, combined with client-server for remote sessions. We use Garlan and Shaw's terminology [18]. WezTerm's documentation does not name any style, so each label below is our interpretation of the evidence cited.

Repository with notification (primary). The Multiplexer is the shared store of session state. Components do not receive screen contents from it; they receive notifications that something changed and then pull what they need. This keeps the GUI independent of how many panes exist or where they run. The cost is that every front end depends on the Multiplexer, which makes it the most critical component to keep correct and fast.

Client-server (primary). The Remote Mux lets a client talk to a multiplexer server over a Unix socket, SSH or TLS, so sessions persist independently of any graphical window. The performance of a remote pane depends on the network and on the versioned protocol between client and server.

Supporting styles. The system is also relaxed-layered (lower layers work without the ones above them, but not in one strict hierarchy), uses implicit invocation (Mux events and Lua event handlers), and uses the interpreter style (Lua configuration and plugins). Pipe-and-filter describes only the output data path, from PTY bytes through the parser to the screen model.

Style	Role in WezTerm	Where it applies
Repository with notification	Primary	The Multiplexer as shared session state
Client-server	Primary	Remote mux server, WezTerm cli
Layered (relaxed)	Supporting	Emulation core, session, front ends
Implicit invocation	Supporting	Mux events, Lua event handlers
Interpreter	Supporting	Lua configuration and plugins
Pipe-and-filter	Partial	The output data path only

The repository model with client-server and layered sub-styles was the only combination that covered both local and remote use, and both the data path and the control path. Its cost is that it needs several styles to describe; we address this by naming the repository as the dominant one.

2.4 Alternative Architectures

To check whether our decomposition is specific to WezTerm, we compared it with four other terminals. The comparison shows that WezTerm's distinguishing decision is to put a tmux-like session server inside a GPU terminal, and to make it scriptable.

System	Architecture	What it shows about WezTerm's choices
Ghostty [8]	A shared core library, libghostty, handles emulation, fonts and rendering; each platform gets its own native GUI (Swift on macOS, GTK4 on Linux).	Ghostty draws its core/GUI boundary at a library and rewrites the GUI per platform. WezTerm writes one GUI over its own cross-platform window layer, trading native look for a single codebase.
kitty [9]	A GPU-based terminal written in C, Python and Go, extended through Python "kittens" and controlled by scripts.	kitty extends through separate programs, WezTerm through an embedded interpreter. kitty has no detachable session server.
tmux [10]	A standalone multiplexer: "tmux keeps all its state in a single main process, called the tmux server", and clients attach and detach.	The closest match to WezTerm's client-server half, but tmux draws inside another terminal. WezTerm feeds its sessions to a native GUI, so scrollback, mouse and clipboard behave natively.
Alacritty [11]	An OpenGL terminal with no tabs or splits; those are "best left to a window manager or terminal multiplexer".	The opposite extreme: no session layer at all. It shows that WezTerm's Multiplexer is a deliberate addition, not a requirement of being a terminal.

2.5 Subsystems

2.5.1 GUI Front End

The GUI is the part of WezTerm that the user sees, clicks and types into. WezTerm keeps track of two separate things: the multiplexer's windows and the GUI windows that display them. A useful analogy is a document and the app that shows it: the Multiplexer is the document and the GUI is the app window, so closing the GUI does not close the terminal if its panes run in a separate mux server. The GUI handles mouse and keyboard input and decides what each key or click should do, manages overlays such as copy mode, quick select, the launcher and the command palette,

shows one window per multiplexer window in the active workspace, and reads its settings from the Lua configuration.

2.5.2 Multiplexer

The Multiplexer keeps a record of everything: which programs are running and where they are. Multiplexing means running many terminals inside one WezTerm, and the Multiplexer is responsible for sorting programs into panes, tabs and windows. It usually runs inside the GUI process and can also run as its own server with no GUI. The PTY & Process Layer, the Terminal Emulation Core and Domains all support it.

2.5.3 Domains

A domain describes where and how a pane's program runs. Locally, WezTerm starts the program through the PTY & Process Layer; for a remote session it connects through a mux server. Other domains include Unix, TLS and WSL. Split panes keep the same domain and working directory because WezTerm remembers the domain of the source pane.

2.5.4 Terminal Emulation Core

The Terminal Emulation Core reads the stream of text and escape sequences produced by programs and turns it into a model of the screen, acting as the translator between programs and the display. Continuing the analogy, it is the writer of each page: every character is placed in a cell of a grid of rows and columns. Its responsibilities are the screen state (the data in each cell), the scrollback, and the encoding of keyboard input into bytes the program can understand.

2.5.5 PTY & Process Layer

This layer runs the programs that WezTerm hosts. WezTerm asks the operating system for a pseudo-terminal, a virtual terminal, and starts a program on it, usually a shell. WezTerm holds the controller end and the program holds the client end; Windows provides its own equivalent, ConPTY. The layer also defines the default program, start folder and environment variables. Domains use it to launch local programs, and each Multiplexer pane monitors its PTY and process. Because PTYs are virtual, one emulator can run many panes without extra hardware, on both Unix and Windows.

2.5.6 Configuration & Lua

WezTerm's settings are written as a Lua program instead of being chosen from a settings menu, which allows a highly customized environment. Other subsystems read from it: for example, the Multiplexer reads the domains and the GUI reads its settings and key tables. Its responsibilities are key assignments (what each command does), event handlers (custom code that WezTerm runs) and plugins (ready-made Lua configurations written by others that add features).

2.5.7 Launcher & CLI

The command line is WezTerm's way of taking commands through typing instead of shortcuts or the mouse. It does not open a window of its own: it asks an instance that is already running to create panes, tabs and programs, and the Multiplexer does the actual work. It decides which multiplexer or GUI to talk to, and always speaks the mux protocol to the mux server. Commands include activate-pane-direction, activate-tab and adjust-pane-size. It cannot press buttons in the GUI or reach overlays such as the command palette.

2.5.8 Renderer and Font Subsystem

WezTerm relies on FreeType and HarfBuzz and maintains its own font fallback list. The GUI uses the Renderer, which draws the content of each pane into windows provided by the Window System, using the GPU where possible.

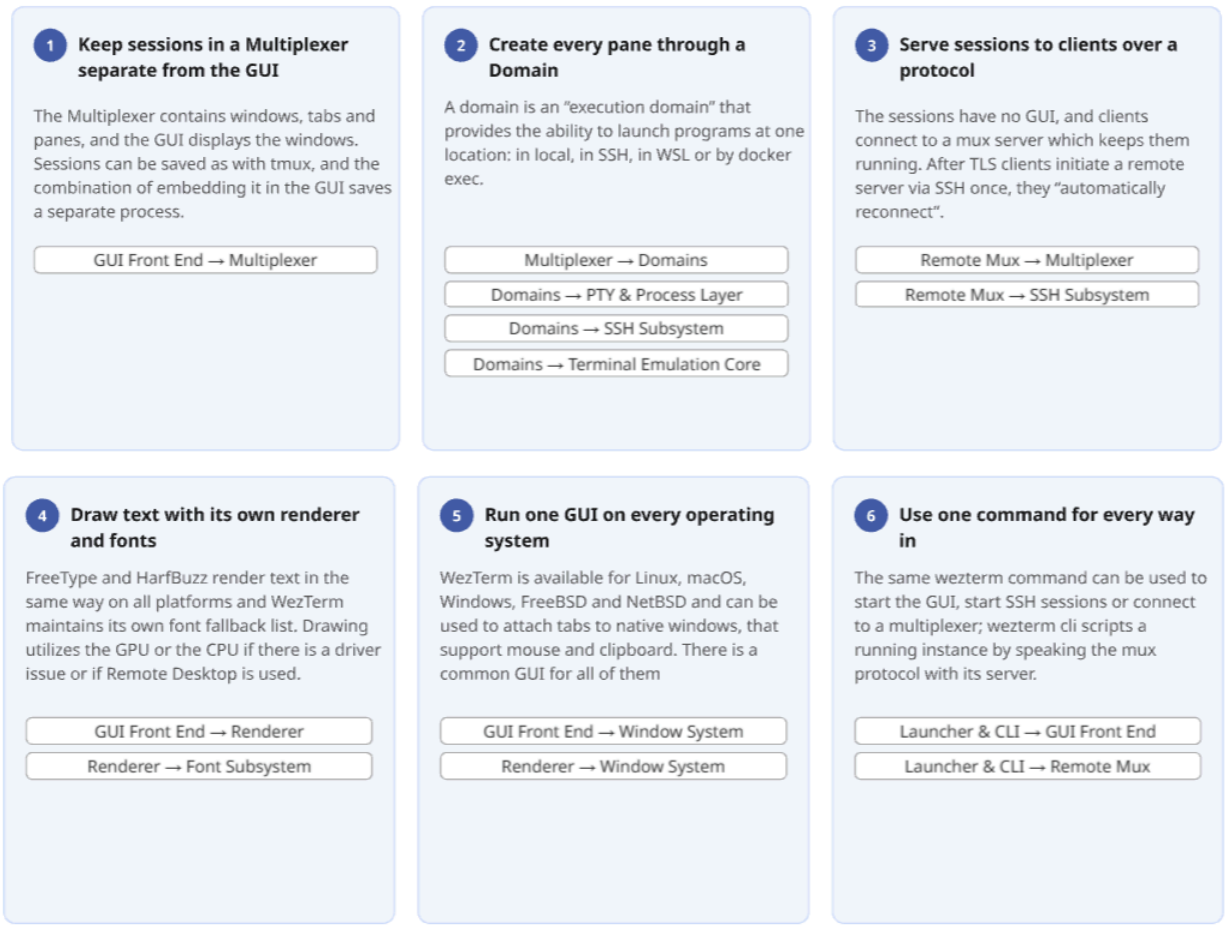
2.5.9 Remote Mux and SSH Subsystem

The Remote Mux keeps sessions alive in a remote mux server without needing a GUI. Clients reach it over Unix sockets, SSH or TLS, and the CLI reaches it with the mux protocol. WezTerm has a built-in SSH client that powers WezTerm ssh and SSH domains, and is also used to start the remote server for TLS sessions, which then reconnect automatically.

2.5.10 Window System

The Window System is WezTerm's adapter to each operating system. It opens real windows, passes on mouse, keyboard and clipboard data, and gives the Renderer a surface to draw on, so that the GUI works on any operating system.

2.5.11 Rationale for Interactions



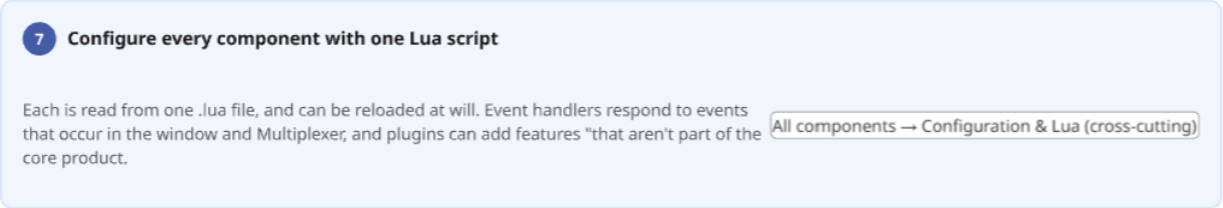


Figure 3. Rationale for the interactions between components.

2.6 Control Flow

2.6.1 Control and Data Flow

Control in WezTerm flows top-down, from user actions to the Multiplexer, while data flows bottom-up, from the shell to the screen. The two meet at the pane. Only change notifications, not screen contents, are pushed upward from the Multiplexer to its subscribers [13].

Input (user to shell). The Window System delivers an operating-system key or mouse event to the GUI Front End. The GUI checks the user's key table, which comes from the Lua configuration. A match triggers a key assignment, such as splitting a pane, opening a tab or running a Lua callback, and the GUI carries it out as a Multiplexer operation [19]. Otherwise the key goes to the active pane, where the Terminal Emulation Core encodes it as an escape sequence and writes the bytes to the PTY, or to the remote server for a pane in a remote domain [2][12].

Output (shell to screen). The shell writes bytes to its PTY, or the remote server forwards them as protocol messages. A reader thread reads the bytes, and a parser thread turns them into terminal actions that are applied to the pane's terminal model. The Multiplexer then publishes a PaneOutput notification. The GUI, which subscribes to the Multiplexer, marks its window as needing repaint; on repaint it pulls the visible lines from the pane, shapes them through the Font Subsystem and draws them with the Renderer. For a remote pane, the server instead pushes only the lines that changed (the protocol defines a GetPaneRenderChanges message [14]).

Configuration. Configuration & Lua is a cross-cutting component. It is read by every other component, reloaded when the file changes, and dispatches events to user callbacks such as update-status and format-tab-title [5]. The Terminal Emulation Core owns what the terminal contains, the Multiplexer owns what sessions exist, and the Renderer only reads state in order to draw it. The Multiplexer never parses escape sequences itself, and none of the front ends talks to a PTY directly.

2.6.2 Units of Concurrency

WezTerm has concurrency at three levels.

- **Processes.** The GUI, the headless WezTerm-mux-server and WezTerm cli run as separate processes that communicate over sockets, using a Unix domain socket, SSH or TLS [3][4]. A mux server runs as a daemon independently of any one GUI.
- **Threads.** Based on our reading of the source, the GUI main thread runs the operating-system event loop, rendering and Lua event callbacks, with an asynchronous executor scheduling work onto it. Each local pane has two further threads: a reader that blocks on the PTY, and a parser that turns the bytes into terminal actions, joined by a socket pair. The reader blocks

because non-blocking PTY reads are not portable to all platforms [13]. The parser batches actions for a short coalescing delay, 3 ms by default, so that large bursts of output do not cause a repaint per chunk [13][21].

- **Asynchronous tasks.** Based on our reading of the source, the mux server handles each client connection as an asynchronous task, multiplexing that client's protocol messages and Multiplexer notifications in one loop. A file watcher also triggers re-evaluation of the Lua configuration when it changes.

Several of these mechanisms protect the performance of the output path, for example when a user prints a very large file. The coalescing delay, cached text shaping and delta-only updates for remote panes all reduce the work done per byte of output.

2.7 Evolution and Division of Responsibilities

The architecture is built to absorb change in a few specific places. New kinds of sessions are added as Domains without touching the Multiplexer, the renderer back end is chosen through configuration (OpenGL, WebGPU or a CPU fallback) [16], new behaviour can be added through Lua event handlers and plugins [5][15], and the client and server of the Remote Mux check protocol compatibility with a versioned codec before exchanging messages [14]. WezTerm itself describes multiplexing as a feature that is still evolving [3], which is consistent with it being the part of the design that sits behind a versioned protocol.

These boundaries also suggest how developer responsibilities could be divided: one group for the front ends (GUI Front End, Launcher & CLI and Window System), one for presentation (Renderer and Font Subsystem), one for sessions (Multiplexer, Domains and Remote Mux), one for terminal emulation and process handling (Terminal Emulation Core and PTY & Process Layer), and one for configuration and scripting. The Multiplexer is the main coordination point, since every front end depends on it, so changes to its notifications need agreement between groups.

3 External Interfaces

Configuration: the user writes a Lua program, WezTerm.lua, and uses the Lua API to define key assignments, events, appearance and plugins [5].

Command line: WezTerm start, ssh, serial, connect and cli are handled by the Launcher & CLI. WezTerm cli interacts with a mux server [17].

Shell and child processes: the PTY & Process Layer is the boundary between WezTerm and the program running in a pane [2].

Terminal escape sequences: programs inside a pane control the terminal through escape sequences, including inline image protocols (iTerm2, kitty and Sixel) [20][1].

Remote hosts: the SSH Subsystem provides WezTerm's built-in SSH client [4], and the Remote Mux connects to multiplexer servers over Unix sockets, SSH and TLS [3].

Operating system and hardware: native windowing (Cocoa, Win32, X11, Wayland), clipboard, system font services, and the GPU through OpenGL or WebGPU (Metal, Vulkan, DirectX 12), with a CPU fallback [16].

4 Use Cases

4.1 Use Case 1: Running a Command in a Local Pane

The first use case follows a user typing a command in a local pane, from the keyboard to the screen (Figure 4).

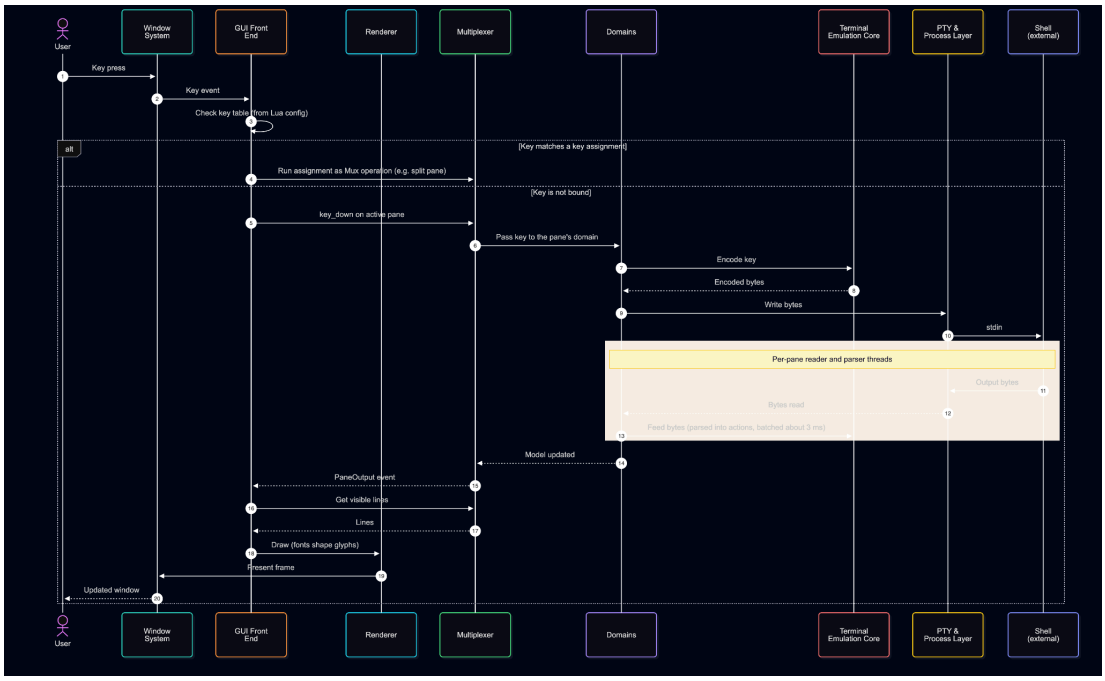


Figure 4. Use case 1: running a command in a local pane.

Legend: participants are the component names used in Figure 1, plus the external Shell and the User. Solid arrows are calls or messages, and dashed arrows are replies or notifications. The numbers show the order of steps. The alt frame separates a key bound to a key assignment from an unbound key. The shaded block marks work done on the pane's background reader and parser threads; everything else runs on the GUI main thread.

The user presses a key, and the Window System hands the event to the GUI Front End. The GUI checks the key table built from the Lua configuration. If the key matches a key assignment, such as a split or a new tab, the GUI turns it into a Multiplexer operation and the shell is not involved. Otherwise, the GUI sends a key_down call for the active pane to the Multiplexer, which passes it to the pane's Domain. The Domain uses the Terminal Emulation Core to encode the key and writes the bytes through the PTY & Process Layer to the shell.

The shell's output returns through the PTY, where the pane's reader and parser threads convert it into terminal actions that update the Terminal Emulation Core's screen model. The Multiplexer then publishes a PaneOutput event. The GUI does not receive the screen contents with this event: it asks the Multiplexer for the visible lines and passes them to the Renderer, which uses the Font Subsystem to shape glyphs, and the frame is presented through the Window System. This use case shows the event-driven notification of the repository style: the GUI learns about new output only through the Multiplexer.

4.2 Use Case 2: Attaching to a Remote Session

The second use case shows WezTerm acting as a multiplexer across machines (Figure 5). The user runs WezTerm connect myhost for an SSH multiplexing domain.

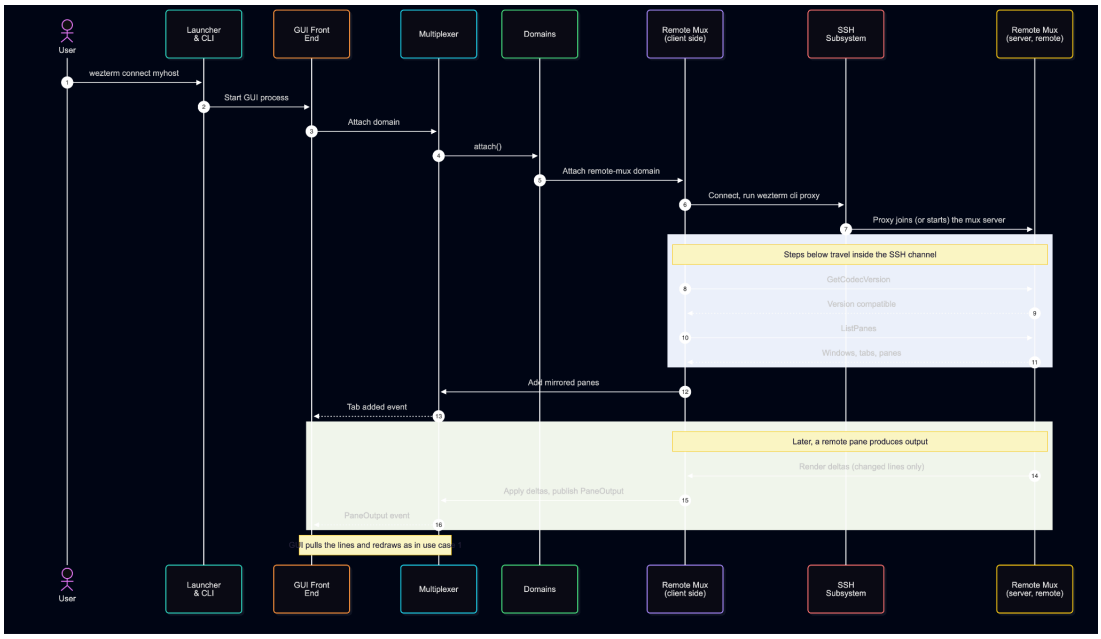


Figure 5. Use case 2: attaching to a remote session over SSH.

Legend: participants are the component names used in Figure 1. “Remote Mux (client side)” and “Remote Mux (server, remote)” are the two halves of the single Remote Mux component. Solid arrows are calls or messages, and dashed arrows are replies or notifications. The blue block marks messages that travel inside the SSH channel, and the green block marks the later push of render updates from the remote server.

The Launcher & CLI starts the GUI process, and the GUI asks the Multiplexer to attach the domain. The Multiplexer asks the Domains component, which hands the work to the client side of the Remote Mux. The client uses the SSH Subsystem to connect to the remote host and to run WezTerm cli proxy there, which joins the mux server or starts it if it is not running [3][22]. Inside that SSH channel, the client first checks protocol compatibility with GetCodecVersion, then asks for the remote panes with ListPanes [14]. It mirrors them into the local Multiplexer, which publishes a tab-added event to the GUI. After this, the server pushes render deltas whenever a remote pane produces output, and the client applies them and publishes PaneOutput as it would for a local pane [13].

The key point of this use case is that the GUI never knows whether a pane is remote. It reacts to the same Multiplexer events as in use case 1, which is the main benefit of the repository style and the evidence for the client-server style in Section 2.3. It also shows two limits: the messages between client and server form a versioned protocol that is not described in the user documentation, and WezTerm describes multiplexing as a feature that is still evolving [3]. Together, these two use cases touch all 12 components in Figure 1, and every interaction they needed was already present in the diagram.

5 Data Dictionary

Pane, tab, window, workspace - The units of terminal organization. A pane holds one running program; panes are arranged in tabs, tabs in windows, and windows in workspaces.

Multiplexer (Mux) - The component that holds the shared state of all windows, tabs, panes and workspaces, and publishes notifications when it changes.

Domain - A source of panes that defines where and how a pane's program runs, such as local, SSH, WSL, Unix, TLS, serial or exec.

Mux server - A headless process that runs a Multiplexer without a GUI so that sessions persist after a GUI window is closed.

Pseudo-terminal (PTY) and ConPTY - A virtual terminal device provided by the operating system that connects WezTerm to a shell. ConPTY is the Windows equivalent.

Escape sequence - A special byte sequence in a program's output that controls the terminal, such as moving the cursor, changing colours or drawing inline images.

Scrollback - The lines that have scrolled off the visible screen but remain stored in the terminal model.

Key assignment - A user-configured rule in the Lua configuration that maps a key or click to an action, such as splitting a pane.

Coalescing delay - A short wait (3 ms by default) during which the parser batches terminal actions so that bursts of output do not trigger a repaint per chunk.

Repository with notification - An architectural style in which components share a central store of state and are told when it changes, rather than being sent the data itself.

Relaxed layered - A layered style in which higher layers use lower layers but are not restricted to the layer directly below, and which does not follow one strict hierarchy.

6 Naming Conventions

API - Application Programming Interface

CLI - Command Line Interface

GUI - Graphical User Interface

GPU - Graphics Processing Unit

Mux - Multiplexer; “mux server” and “mux protocol” refer to the Multiplexer running as a server and its client-server protocol

PTY - Pseudo-terminal; ConPTY is the Windows equivalent

SSH - Secure Shell; TLS - Transport Layer Security; WSL - Windows Subsystem for Linux

Component names are capitalized as in Figure 1 (for example, Terminal Emulation Core); code identifiers, commands and message names (for example, PaneOutput, WezTerm cli) appear as written in WezTerm.

7 Conclusion

This report developed a conceptual architecture of WezTerm to explain how its major components cooperate to support terminal emulation, session management, customization and remote access. By examining the official documentation, repository structure, package dependencies and two representative use cases, we identified 12 components and their principal responsibilities. The Multiplexer is the central component: it maintains a shared session state and notifies interested components of changes, which lets the GUI Front End, the Launcher & CLI and the Remote Mux interact with terminal sessions through a common model.

The two use cases validated this architecture by showing how control and data flow through it. Local command execution exercises the GUI, Multiplexer, Domains, terminal emulation and process management, while remote attachment shows how the Remote Mux and SSH Subsystem integrate remote sessions into the same interface. These scenarios support our classification of WezTerm as primarily a repository with change notification, complemented by a client-server architecture and elements of a relaxed layered design. The comparison with Ghostty, kitty, tmux and Alacritty highlights WezTerm's distinctive choice of integrating terminal emulation, multiplexing and remote-session support.

Our model has limitations. WezTerm does not provide a definitive architectural specification, so the component boundaries and style classifications are our interpretation, and grouping more than 60 Rust packages into 12 components improves readability but omits internal distinctions and some complex dependencies. We examined selected concurrency mechanisms and two use cases, but did not perform performance profiling, controlled experiments or cross-platform testing, so other domains, configuration callbacks, error conditions and reconnection behaviour may introduce interactions that our diagrams do not capture. Further work could strengthen the model through more source-code tracing, runtime experiments and validation with WezTerm's maintainers.

8 Lessons Learnt

One of the main lessons was the importance of distinguishing conceptual architecture from implementation details. WezTerm contains more than 60 Rust packages, but presenting each one would have made the architecture difficult to understand. Grouping related packages into 12 components allowed us to focus on meaningful responsibilities and interactions instead of mirroring the repository structure.

We also learned the value of validating architectural decisions against concrete use cases. Tracing local command execution and remote session attachment helped us identify gaps and refine our proposed relationships. Additionally, architectural styles are not always mutually exclusive: WezTerm could not be adequately described by a single traditional style, so combining repository with change notification, client-server and relaxed layered classifications gave a more complete explanation.

Finally, our use of AI tools showed both their value and their limitations. AI accelerated repository exploration, sequence diagram generation and proofreading, but sometimes introduced incorrect components or dependencies. Cross-checking its outputs against source code,

dependency information and documentation reinforced the importance of independent verification, and we learned to use AI for exploration and repetitive work while keeping human responsibility for architectural decisions.

9 References

[1] WezTerm. Features. https://WezTerm.org/features.html

[2] WezTerm. What is a Terminal? https://WezTerm.org/what-is-a-terminal.html
[3] WezTerm. Multiplexing. https://WezTerm.org/multiplexing.html
[4] WezTerm. SSH. https://WezTerm.org/ssh.html

[5] WezTerm. Configuration (Lua) reference. https://WezTerm.org/config/files.html
[6] WezTerm. Source repository. https://github.com/WezTerm/WezTerm
[7] WezTerm. Contributing guide. https://WezTerm.org/contributing.html

[8] Ghostty. About Ghostty. https://ghostty.org/docs/about

[9] kitty. Overview. https://sw.kovidgoyal.net/kitty/overview/

[10] tmux. Getting Started (wiki). https://github.com/tmux/tmux/wiki/Getting-Started

[11] Alacritty. Source repository and README. https://github.com/alacritty/alacritty

[12] WezTerm. term crate (terminal model). https://github.com/WezTerm/WezTerm/tree/main/term
[13] WezTerm. mux crate (Multiplexer). https://github.com/WezTerm/WezTerm/tree/main/mux
[14] WezTerm. codec crate (mux protocol). https://github.com/WezTerm/WezTerm/tree/main/codec
[15] WezTerm. Plugins. https://WezTerm.org/config/plugins.html
[16] WezTerm. front_end option. https://WezTerm.org/config/lua/config/front_end.html
[17] WezTerm. Command line interface (WezTerm cli). https://WezTerm.org/cli/cli/index.html
[18] Garlan, D. and Shaw, M. An Introduction to Software Architecture. Carnegie Mellon University, CMU-CS-94-166, 1994.
[19] WezTerm. Key assignments. https://WezTerm.org/config/keys.html

[20] WezTerm. Image protocols (imgcat). https://WezTerm.org/imgcat.html

[21] WezTerm. Mux_output_parser_coalesce_delay_ms. https://WezTerm.org/config/lua/config/mux_output_parser_coalesce_delay_ms.html

[22] WezTerm. WezTerm cli proxy. https://WezTerm.org/cli/cli/proxy.html
[23] CISC 322/326 Software / Game Architecture. Term Project guidelines.

10 AI Collaboration Report

10.1 AI Member Profile and Selection Process

Our team integrated a multi-agent AI suite as its virtual teammate: Claude Sonnet 5.5 (Anthropic), ChatGPT-5.6 Luna (OpenAI), Cursor Pro (Anysphere) and Claude Code (Anthropic's terminal-based coding agent). We selected tools by matching capabilities to the needs of Assignment 1. The standard web LLMs (Claude Sonnet 5.5 and ChatGPT-5.6 Luna) were chosen for natural-language summarization and report structuring, since much of A1 involves condensing documentation into coherent prose. Cursor Pro and Claude Code were chosen because the target system's codebase is too large to paste into a chat window: both can securely index, search and analyze a local repository, which let us ask targeted questions without

copying thousands of lines of code and without the truncation and context loss that pasting causes. We considered using a single general-purpose chatbot for everything but rejected it, because it could not see the repository and would have pushed us toward answering codebase questions from the model's general knowledge, which is where hallucination risk is highest.

10.2 Tasks Assigned to the AI Teammate

1) **Generating PlantUML code for sequence diagrams.** Rationale: AI is strong at translating human-described logic flow into strict UML syntax, which saved us formatting time and let us focus on deciding what the diagrams should show.
2) **Querying the codebase for global control flow and concurrency patterns using Claude Code.** Rationale: the terminal agent can rapidly search a large repository for threading libraries, entry points and event loops, which would take a human team hours of manual searching.
3) **Proofreading and standardizing the tone of the final report using ChatGPT-5.6 Luna.** Rationale: the report was written by six different members, and a language model is well suited to enforcing a unified voice and consistent terminology.

We deliberately did not delegate the conceptual mapping of components, the choice of architectural styles or the analysis of developer responsibilities, since these are the core judgments the assignment evaluates.

10.3 Interaction Protocol and Prompting Strategy

Our group designated one member as the “Lead AI Integrator”. This person ran all substantive AI sessions, maintained a shared log of prompts and outputs, and relayed requests from teammates, who submitted requests that the Integrator refined before running them. We chose this over six siloed chats to preserve context continuity, avoid contradictory outputs from sessions with different assumptions, and keep a single audit trail of what the AI produced.

Our login sequence diagram shows how prompts evolved. The first attempt was: “Make a sequence diagram for logging in.” This returned a generic diagram with invented components that did not match our system. We then added context and a persona: “Act as a software architect. Using this specific login function from our repository, generate a PlantUML sequence diagram.” This anchored the output in real code but still included low-level detail. Our final prompt added constraints: “Act as a software architect. Using this specific login function from our repository, generate a PlantUML sequence diagram. Constrain your output to only the top-level subsystems. Do not include database implementation details.” This produced a diagram at the right level of abstraction for a conceptual architecture. Each refinement came from specifying context, role, scope and exclusions; vague prompts produced plausible but ungrounded output.

10.4 Validation and Quality Control Procedures

We treated all AI output as unverified until it passed a three-step workflow.

Step A: Dependency cross-referencing. Every AI-generated claim about subsystem

dependencies was checked against the dependency matrix we extracted using the Understand tool. Any dependency the AI asserted that did not appear in the matrix was flagged, and any dependency in the matrix that the AI omitted was investigated.

Step B: Manual tracing against source code. A human member traced each AI-generated sequence diagram message by message against the primary source code. This caught a hallucination: in one draft, the AI depicted a direct call from the UI to the Database, whereas tracing the code showed that all data access is mediated by the Controller. A human corrected the diagram to route the interaction through the Controller, and we recorded the error in our prompt log as a known failure mode to check for in later diagrams.

Step C: Peer review of AI-generated text. All AI-written or AI-edited text was reviewed by at least one human member other than the original author of the section, checking for factual accuracy, clarity and academic tone, and confirming that proofreading passes had not altered technical meaning or terminology.

10.5 Quantitative Contribution to Final Deliverable

We estimate the AI's overall contribution to the final deliverable at **22%**: 10% from accelerating codebase exploration (control flow and concurrency queries), 7% from UML generation (PlantUML drafting and syntax) and 5% from formatting and proofreading. The core architectural logic, the conceptual mapping of components, the identification of architectural styles and the analysis of responsibilities remained 100% human-driven. The AI accelerated how quickly we reached and presented our conclusions, but it did not decide what those conclusions were. We arrived at the estimate by reviewing our prompt log and the share of the final report that each AI output touched after human revision.

10.6 Reflection on Human-AI Team Dynamics

Integrating the AI saved hours on boilerplate diagramming and syntax, but it also created new work in the form of rigorous validation and prompt engineering. Net time savings were real but smaller than they first appeared, because every output required verification before we could rely on it. The AI also influenced our brainstorming: it rapidly proposed alternative architectural styles, which the team then debated, defending or rejecting each against the evidence in the documentation. It was most useful as a source of options to argue against, not as a decision-maker.

One challenge arose early. The group initially over-relied on the AI's structural assumptions, which led to a brief disagreement over subsystem boundaries: some members trusted the AI's proposed decomposition while others pointed to inconsistencies with the documentation and code. Resolving it taught the team to trust the source code over the LLM's output. Going forward, we will treat AI as a fast but fallible collaborator: use it to generate drafts and explore options, require evidence before accepting any factual claim, keep a single integrator and prompt log for accountability, and reserve the central architectural reasoning for humans.